

PROBLEM 1

Demand Forecasting

Predicting future demand to optimize inventory and supply chains

ML Interview Preparation Guide

What Is Demand Forecasting?

Demand forecasting means predicting how much of something people will buy in the future so you can prepare the right amount. If you run a grocery store and order too little bread, shelves go empty and you lose sales. Order too much, and bread goes stale and you waste money.

Amazon does this for millions of products. Netflix does this for server capacity. Uber does this for how many drivers to have ready at any given time.

Real World Examples: Retail inventory replenishment, restaurant food prep quantities, e-commerce warehouse stocking, ride-sharing driver allocation, server capacity planning.

Step 0: Clarify the Problem First

In an interview, the first thing that impresses people is NOT jumping to solutions. Always ask clarifying questions before touching data or models.

Key Questions to Ask

- **What are we predicting?** Units sold? Revenue? Inventory needed?
- **What is the granularity?** Per SKU, per store, per region?
- **What is the forecast horizon?** Next day? Next week? Next quarter?
- **What is the update frequency?** Retrain daily? Weekly?
- **What is the cost of error?** Stockout (lost revenue) vs overstock (holding cost)? This determines your loss function.
- **Is this cold-start or warm?** New products with no history?

Example framing: "I will assume we are forecasting daily unit sales per SKU per store, 7 days ahead, for a retail chain with ~10,000 SKUs."

Step 1: Data Collection and EDA

Data Sources to Pull In

Source	Features
Historical sales	Target variable — units sold per day
Inventory records	Stockouts (if inventory = 0, sales are censored)
Promotions/pricing	Discount %, promo flags, price elasticity signals

Source	Features
Calendar	Day of week, holidays, payday cycles
Weather	Temperature, rain — critical for food/apparel
Macroeconomic	CPI, unemployment — for longer horizons
Web/search trends	Google Trends as a leading indicator

Key EDA Questions

- **Is the data stationary?** Use the ADF test (Augmented Dickey-Fuller). If $p < 0.05$, series is stationary. If not, you need differencing.
- **Seasonality?** Use STL decomposition (Seasonal + Trend + Residual). Check ACF/PACF plots.
- **Stockout periods?** Sales = 0 on days where inventory = 0 is censored data, not true zero demand. You must impute or model it separately.
- **Distribution of target?** Usually right-skewed. Consider log-transform.

Understanding Stationarity

A time series is stationary if its mean, variance, and autocovariance are time-invariant — the behavior does not change over time.

- **Stationary** = calm lake — waves are similar every day
- **Non-stationary** = river with a current — constantly drifting somewhere

Fix non-stationarity via differencing: $y'_t = y_t - y_{t-1}$. The ADF Test checks for this: reject H_0 ($p < 0.05$) means stationary.

ACF and PACF Plots

ACF (Autocorrelation Function): Does today's sales correlate with sales from k days ago? A spike at lag 7 means weekly seasonality — add "sales from 7 days ago" as a feature.

PACF (Partial ACF): Same idea but removes in-between effects. Used to pick ARIMA model parameters.

Step 2: Feature Engineering

Feature engineering is where interviews are won or lost. Go beyond the obvious. Models don't understand "Monday" or "Christmas" — you must convert everything into numbers.

Temporal Features

- `day_of_week` (0–6), `is_weekend`, `is_holiday`
- `day_of_month`, `week_of_year`, `month`, `quarter`
- `days_until_holiday`, `days_after_holiday` — very powerful signal
- `is_payday` (1st/15th of month)

The Cyclical Encoding Trick

If you encode Monday=0 and Sunday=6, the model thinks Sunday and Monday are far apart (distance of 6). But they are neighbors! Fix this with sine/cosine encoding:

- `day_sin = sin(2π × day / 7)`
- `day_cos = cos(2π × day / 7)`

Now Monday and Sunday are mathematically close in a circle. This uses the unit circle from trigonometry.

Lag Features — The Most Important

Use the past to predict the future. If last Monday we sold 150 loaves, this Monday will probably be similar.

- `lag_1` = sales from yesterday
- `lag_7` = sales from 7 days ago (same day last week)
- `lag_14` = sales from 14 days ago
- `lag_28` = sales from 28 days ago (same day last month)

Rolling Statistics

- `rolling_mean_7d` = average sales over last 7 days (captures trend)
- `rolling_mean_28d` = average sales over last 28 days
- `rolling_std_7d` = how much sales fluctuated last week (captures volatility)
- `ewm_alpha_0.3` = exponentially weighted mean (recent days matter more)

Interaction Features

- `price × promo_flag` — promo effectiveness varies by price point
- `sku_category × day_of_week` — beer sells more on Fridays, not Mondays
- `store_region × weather` — rain affects outdoor stores differently

Step 3: Statistics and Math

Loss Functions and Their Implications

Loss Function	Formula	When to Use
MAE	$\text{mean}(y - \hat{y})$	Symmetric cost, robust to outliers
RMSE	$\sqrt{\text{mean}((y - \hat{y})^2)}$	Penalizes large errors — use if big errors are catastrophic
MAPE	$\text{mean}(y - \hat{y} / y)$	Scale-free, but undefined at $y = 0$
sMAPE	Symmetric version	Better than MAPE for zero-sales SKUs
Quantile Loss	$\tau \cdot \max(y - \hat{y}, 0) + (1 - \tau) \cdot \max(\hat{y} - y, 0)$	When you want P90 forecast for safety stock

Asymmetric loss insight: If stockouts are more costly than overstock, use quantile loss with $\tau > 0.5$, which penalizes under-prediction more.

Key Probability Distributions for Sales

- **Poisson:** Good for rare events (specialty items). Mean and variance are equal.
- **Negative Binomial:** When sales are bursty — variance is higher than mean. More realistic.
- **Zero-Inflated Poisson (ZIP):** For items with many true zero-demand days.

Step 4: Models — From Simple to Complex

1. Moving Average (Simplest Baseline)

Forecast for tomorrow = Average of last 7 days. Always start with this. If your fancy model cannot beat this, something is wrong.

2. Exponential Smoothing / Holt-Winters

Tracks three things simultaneously: Level (current average), Trend (going up or down?), and Seasonality (weekly pattern). Combines all three: Forecast = Level + Trend × horizon + Seasonal factor.

- **Pros:** Simple, fast, works well for clean data with obvious seasonality
- **Cons:** Cannot use extra features like promotions or weather

3. ARIMA — Using Your Own Past to Predict Yourself

ARIMA(p, d, q) has three components:

- **AR (AutoRegressive), order p:** predict using past p values
- **I (Integrated), order d:** applies differencing to make data stationary
- **MA (Moving Average), order q:** correct using past error terms

SARIMA extends this with seasonal terms (P, D, Q, m) where m = seasonal period (7 for weekly).

- **Pros:** Statistically rigorous, interpretable
- **Cons:** One model per store/product (does not scale), cannot use external features

4. Facebook Prophet

Decomposes the series: $y(t) = \text{trend}(t) + \text{seasonality}(t) + \text{holiday_effect}(t) + \text{noise}$. Seasonality is modeled using Fourier series — adding sine and cosine waves of different frequencies.

- **Pros:** Easy to add holidays, handles missing data, interpretable
- **Cons:** Still one model per product, no custom features like weather

5. LightGBM / XGBoost — The Industry Workhorse

The most commonly used model in production forecasting. Key insight: reframe time series as a regular ML problem. Each (store, product, date) combination becomes one row in a table with lag features. Now it is just a regression problem!

- **Pros:** One model for ALL stores/products (global model), uses any features, fast, interpretable via SHAP
- **Cons:** Needs manual lag feature construction, cannot extrapolate beyond seen patterns

6. Deep Learning Models

LSTM (Long Short-Term Memory)

Like a human reading a book — remembers context from chapter 1 even in chapter 10. LSTM has a memory cell that learns what to remember and what to forget over long sequences. GRU is a simplified LSTM with fewer parameters and similar performance.

- **Pros:** Captures long-range patterns without manual lag engineering
- **Cons:** Slow to train, needs lots of data, hard to debug

Temporal Fusion Transformer (TFT) — State of the Art

Uses the attention mechanism to jump to any past time step and focus on the most relevant parts. Handles three input types: static (store location), known future (holidays), and past-only (actual sales). Also outputs quantile forecasts — P10, P50, P90 predictions.

- **Pros:** Most accurate for complex multi-step forecasting, interpretable attention weights, probabilistic outputs
- **Cons:** Complex to set up, needs significant data

DeepAR (Amazon)

Instead of predicting a single number, DeepAR predicts a probability distribution over future sales. Learns from all products at once (global model) and handles cold start by borrowing patterns from similar products.

Model Comparison Table

Model	Scales to 1000s of Products?	External Features?	Uncertainty Estimates?	Needs Lots of Data?
ARIMA	No	No	Yes	No
Prophet	No	Partial	Yes	No
LightGBM	Yes	Yes	Partial	No
LSTM	Yes	Yes	No	Yes
TFT	Yes	Yes	Yes	Yes
DeepAR	Yes	Yes	Yes	Yes

Step 5: Evaluation — The Critical Mistake to Avoid

NEVER do random train/test split on time series! If your data is Jan–Dec and you randomly pick 20% for testing, your model will train on December data and test on January data. It is cheating — it saw the future during training.

Walk-Forward Validation

Always use walk-forward (time-series cross-validation). This mimics reality — you always train on the past and predict the future.

- Round 1: Train on Jan–Sep → Test on Oct
- Round 2: Train on Jan–Oct → Test on Nov
- Round 3: Train on Jan–Nov → Test on Dec

Step 6: Putting It in Production

The Daily Pipeline

1. New sales data arrives from store POS systems
2. Feature pipeline runs — compute lag features, add calendar features
3. Model runs inference — outputs daily forecast per store per product
4. Results stored in a database (BigQuery/Redis)
5. Replenishment system reads forecasts and orders inventory

Training-Serving Skew — The Biggest Production Bug

Your model was trained with features computed one way, but in production you compute them slightly differently. The model gets confused and performance drops mysteriously.

Fix: Use a Feature Store — a central system (Feast, Tecton) that computes and stores features once, used by both training and serving.

Retraining Strategy

- **Scheduled retraining:** Weekly or monthly. Simple and predictable.
- **Triggered retraining:** When monitoring detects drift — more adaptive.
- **Concept drift:** Demand patterns change (new competitor, economic shift). Detect via rolling forecast error vs. baseline.

Monitoring in Production

- **Data drift:** KL divergence or PSI (Population Stability Index) on feature distributions
- **Prediction drift:** Track MAPE week-over-week
- **Anomaly detection on forecasts:** If forecast for SKU spikes 10x vs last week, flag for human review

Handling Cold-Start (New SKUs)

- Use global model (LightGBM or DeepAR) with SKU metadata features (category, price tier, supplier)
- Find similar SKUs via cosine similarity on metadata embeddings and use their historical patterns

How to Answer in an Interview

Before anything, clarify what we are forecasting, at what granularity, and what the cost of over vs. under-prediction is — because that determines our loss function. For data, use historical sales, promotions, calendar features, and weather. The trickiest data issue is stockouts — days with zero sales because we ran out, not because no one wanted to buy. For features, the most important are lag features (7, 14, 28 days ago) and rolling means. Start with LightGBM global model as a strong baseline. Evaluate using walk-forward validation — never random split. For production: daily batch pipeline with feature store, weekly retrains, and monitoring for forecast drift.